

---

**Algorithm 1** Shampoo<sup>2</sup> [gupta2018shampoo,morwani2024new] for  $W \in \mathbb{R}^{d_a \times d_b}$ .

---

**Require:** learning rate  $\gamma$ , momentum  $\beta_1$ , EMA decay  $\beta_2$ , refresh period  $T$ , damping  $\varepsilon$

**State:** momentum  $M$ , full SPD factors  $L, R$ .

- 1:  $M \leftarrow 0, \quad L \leftarrow \varepsilon I_{d_a}, \quad R \leftarrow \varepsilon I_{d_b}$
  - 2: **For each step  $t$  with gradient  $G \in \mathbb{R}^{d_a \times d_b}$ :**
  - 3:  $M \leftarrow \beta_1 M + (1 - \beta_1)G$   $\triangleright$  momentum
  - 4:  $L \leftarrow (1 - \beta_2)L + \beta_2 G G^\top$   $\triangleright$  raw row-Gram EMA
  - 5:  $R \leftarrow (1 - \beta_2)R + \beta_2 G^\top G$   $\triangleright$  raw col-Gram EMA
  - 6: **Every  $T$  steps:** factor  $L^{-1/4} = Q_L \text{diag}(\Lambda_L)^{-1/4} Q_L^\top$ ,
  - 7:  $R^{-1/4} = Q_R \text{diag}(\Lambda_R)^{-1/4} Q_R^\top$   $\triangleright$  full eigendecomp
  - 8:  $\theta \leftarrow \theta - \gamma L^{-1/4} M R^{-1/4}$   $\triangleright (L \otimes R)^{-1/2} \text{vec}(M)$  in matrix form
- 

## 1 Width-dependent initialization in KL-Shampoo

KL-Shampoo [lin2025klshampoo] maintains a Kronecker-factored matrix preconditioner  $S = S_a \otimes S_b$  for each weight tensor  $W \in \mathbb{R}^{d_a \times d_b}$ , with eigendecomposition  $S_k = Q_k \text{diag}(\lambda_k) Q_k^\top$  for  $k \in \{a, b\}$ . The update is  $\Delta\theta = -\gamma Q \text{diag}(\lambda_a \otimes \lambda_b)^{-1/2} Q^\top \text{vec}(M)$  (lin2025klshampoo, Box on Practical KL-Shampoo).

Implementations [lin2025klshampoo,klshampoojax] store  $\text{esi}_k := \lambda_k^{\odot -1/2}$  directly rather than  $\lambda_k$ , since  $\text{esi}_k$  is what enters the update multiplicatively. Concretely, every  $\lambda^{\odot -1/2}$  appearing in Algorithm 2 (lines 7, 8, and 11) is stored and read as  $\text{esi}$  in the `eigen_sqrt_inv` field of the optimizer state. The buggy initialization on line 4 sets  $\lambda_k(0) = \rho \mathbf{1}$ , which the implementation materializes as  $\text{esi}_k(0) = (1/\sqrt{\rho}) \mathbf{1}$  (`_core.py:101` — the line we patch in Section 1.4). The initialization of  $\text{esi}_k$  at step 0 is what we focus on below.

### 1.1 Background: vanilla Shampoo and the KL-Shampoo modifications

KL-Shampoo is a modification of *Shampoo* [gupta2018shampoo], which maintains two SPD matrices — a left factor  $L \in \mathbb{R}^{d_a \times d_a}$  and a right factor  $R \in \mathbb{R}^{d_b \times d_b}$  — and applies their (matrix) inverse roots to the gradient. Algorithm 1 is the  $e_L = e_R = 1/2$  variant (Shampoo<sup>2</sup>, morwani2024new), which matches KL-Shampoo’s update exponent.

**KL-Shampoo’s four modifications.** Compared with Algorithm 1, KL-Shampoo [lin2025klshampoo] changes the following four pieces, shown in blue in Algorithm 2:

1. **Whitened-EMA target.** The idealized KL-Shampoo updates the row-factor by EMA-ing the *whitened* outer product  $\frac{1}{d_b} G S_b^{-1} G^\top$  (and symmetrically  $S_b$  uses  $\frac{1}{d_a} G^\top S_a^{-1} G$ ). These are MLEs that minimize  $\text{KL}(\mathcal{N}(0, \mathbb{E}[gg^\top] + \kappa I) \parallel \mathcal{N}(0, S))$  (lin2025klshampoo, Eq. 3). In the practical form (Algorithm 2) the whitening enters *at the eigenvalue-EMA level* (line 9), not at the matrix level — but the statistical content is the same.
2. **Factored state  $(Q_k, \lambda_k)$  instead of full  $L, R$ .** Rather than storing the  $d \times d$  SPD factors and re-eigendecomposing them periodically (line 7 of Algorithm 1), KL-Shampoo maintains the eigenbasis  $Q_k$  and eigenvalue vector  $\lambda_k$  separately at all times. This lets it EMA the eigenvalues directly (line 9 of Algorithm 2).

---

**Algorithm 2** KL-Shampoo update for a weight matrix  $W \in \mathbb{R}^{d_a \times d_b}$ . Blue lines mark the modifications vs Shampoo<sup>2</sup> (Algorithm 1); the red line is the source of imperfect width transfer.

---

**Require:** learning rate  $\gamma$ , momentum  $\beta_1$ , EMA decay  $\beta_2$ , QR refresh period  $T$ , **initial eigenvalue scalar  $\rho$  (default 0.1)**

**State (per parameter  $W$ ):** momentum  $M$ , **eigenbases  $Q_a, Q_b$ , eigenvalues  $\lambda_a, \lambda_b$**  (rather than full  $L, R$ ), gradient-Gram EMAs  $GG_a, GG_b$ .

- 1: **Initialization (first batch with gradient  $G \in \mathbb{R}^{d_a \times d_b}$ ):**
  - 2:  $GG_a \leftarrow \frac{1}{d_b} GG^\top, \quad GG_b \leftarrow \frac{1}{d_a} G^\top G$
  - 3:  $Q_a \leftarrow \text{eigh}(GG_a), \quad Q_b \leftarrow \text{eigh}(GG_b)$  ▷ eigenbases from data
  - 4:  $\lambda_a \leftarrow \rho \cdot \mathbf{1}_{d_a}, \quad \lambda_b \leftarrow \rho \cdot \mathbf{1}_{d_b}$  ▷ eigenvalues NOT from data — width-blind constant
  - 5: **For each subsequent step  $t$ :**
  - 6:  $M \leftarrow \beta_1 M + (1 - \beta_1) G$  ▷ momentum  
// Eigenvalue EMA in the (possibly stale) eigenbasis  $(Q_a, Q_b)$ :
  - 7:  $\hat{G} \leftarrow Q_a^\top G Q_b \text{diag}(\lambda_b^{\odot -1/2})$  ▷ whiten by  $S_b$  side
  - 8:  $d_a^{(t)} \leftarrow \text{mean}(\hat{G}^{\odot 2}, \text{axis} = 1), \quad d_b^{(t)} \leftarrow \text{mean}(\hat{G}'^{\odot 2}, \text{axis} = 0)$  ▷  $\hat{G}'$  uses  $\lambda_a^{-1/2}$
  - 9:  $\lambda_a \leftarrow (1 - \beta_2) \lambda_a + \beta_2 d_a^{(t)}, \quad \lambda_b \leftarrow (1 - \beta_2) \lambda_b + \beta_2 d_b^{(t)}$  ▷ per-step eigenvalue EMA  
// Basis refresh every  $T$  steps:
  - 10: **if**  $t \bmod T = 0$ :  $Q_k \leftarrow \text{QR}(S_k Q_k)$  for  $k \in \{a, b\}$  ▷ power iter., not full eigh  
// Apply preconditioned update:
  - 11:  $\theta \leftarrow \theta - \gamma Q_a \text{diag}(\lambda_a^{\odot -1/2} \otimes \lambda_b^{\odot -1/2}) Q_a^\top M Q_b Q_b^\top$
- 

3. **Eigenvalue EMA in a possibly-stale basis.** Each step,  $\lambda_k$  is updated by EMA-ing the squared whitened projection  $(Q_a^\top G Q_b \text{diag}(\lambda_b^{-1/2}))^{\odot 2}$  along the *current* (potentially out-of-date) basis  $(Q_a, Q_b)$ . Vanilla Shampoo has no per-step eigenvalue update — it EMAs  $L, R$  and re-eigendecomposes every  $T$  steps.

4. **QR power-iteration refresh for  $Q_k$ .** Every  $T$  steps, KL-Shampoo refreshes the basis by  $Q_k \leftarrow \text{QR}(S_k Q_k)$  (one power-iteration step + Gram-Schmidt), *not* a full eigendecomposition of  $L, R$ . This costs  $O(d^2 r)$  per factor for rank- $r$  tracked spectrum, vs.  $O(d^3)$  for eigh.

The net effect is roughly SOAP-comparable per-step cost [vyas2024soap, lin2025klshampoo] without needing Adam grafting: every algorithmic ingredient that previously required grafting (whitening the gradient, refreshing the basis, smoothing the spectrum) is built in.

## 1.2 The algorithm with the offending line highlighted

Algorithm 2 reproduces the practical KL-Shampoo update [Box “Practical KL-Shampoo”]lin2025klshampoo, with the width-dependent failure mode in red.

## 1.3 Why the constant $\lambda_k(0) = \rho \mathbf{1}$ breaks width transfer

Under  $\mu\text{P}$  parameterization, the per-entry variance of the hidden-weight gradient is  $\text{Var}(G_{ij}) = \Theta(1/d_{\text{in}})$  [yang2021v, qiu2026matrixpreconditioned]. The row-Gram averaged by the opposite di-

---

**Algorithm 3** Two width-aware initializations for  $\lambda_k(0)$ . Replace line 4 of Algorithm 2 with one of the lines in **red**.

---

- 1: **Option A (data-driven, recommended):** use the actual first-batch Gram diagonal,
  - 2:    $\lambda_a \leftarrow \text{diag}(\text{GG}_a) + \varepsilon \mathbf{1}_{d_a}, \quad \lambda_b \leftarrow \text{diag}(\text{GG}_b) + \varepsilon \mathbf{1}_{d_b}$   $\triangleright \varepsilon$  tiny for numerical safety
  - 3: **Option B (parametric, simpler):** scale the constant with the dimension,
  - 4:    $\lambda_a \leftarrow \frac{\rho_0}{d_a} \mathbf{1}_{d_a}, \quad \lambda_b \leftarrow \frac{\rho_0}{d_b} \mathbf{1}_{d_b}$   $\triangleright \rho_0$  tuned once at the base width
- 

mension (line 2 of Algorithm 2) therefore satisfies

$$[\text{GG}_a]_{ii} = \frac{1}{d_b} \sum_j G_{ij}^2 \sim \frac{1}{d_b} \cdot d_b \cdot \frac{1}{d_{\text{in}}} = \Theta\left(\frac{1}{d_{\text{in}}}\right), \quad (1)$$

and the equilibrium eigenvalue spectrum scales as  $\lambda_k^* \sim \Theta(1/d)$ , giving  $\text{esi}_k^* = (\lambda_k^*)^{-1/2} \sim \Theta(\sqrt{d})$ . The eigenvalue EMA in line 9 equilibrates over  $\tau_{\text{eq}} \approx 1/(1 - \beta_2)$  steps (typically  $\beta_2 = 0.99$ , so  $\tau_{\text{eq}} \approx 100$ ).

The fixed initialization  $\lambda_k(0) = \rho \cdot \mathbf{1}$  gives  $\text{esi}_k(0) = 1/\sqrt{\rho}$ , which is *width-independent*. The ratio between init and equilibrium is therefore

$$\frac{\text{esi}_k(0)}{\text{esi}_k^*} \propto \frac{1}{\sqrt{\rho \cdot d}} \xrightarrow{d \rightarrow \infty} 0. \quad (2)$$

At wide  $d$  the initial preconditioner is *much* weaker than the equilibrium preconditioner. During the first  $\sim \tau_{\text{eq}}$  steps the optimizer behaves closer to Adam-on-raw-gradients; afterwards the preconditioner “wakes up” and amplifies updates along the small-eigenvalue directions. The magnitude of this wake-up transient grows with  $d$ , so a peak learning rate tuned at the base width over-shoots at wider widths. Empirically the optimal  $\gamma$  drifts downward by a factor of  $\sim 5\times$  across an  $8\times$  width range (Phase 7v2 sweep, see Sec. ?? or PHASE7V2\_REPORT.md in the supplementary material).

## 1.4 Fix: make the initialization width-aware

Two principled replacements for line 4 of Algorithm 2 both restore flat learning-rate transfer; see Algorithm 3.

**Why each was expected to work.** Option A initializes  $\lambda_k(0)$  to the *exact* first-batch estimate of the row/column Gram diagonal — the same quantity the EMA in line 9 is converging toward — so  $\text{esi}_k(0) \approx \text{esi}_k^*$  at every width by construction. Option B preserves the fixed-scalar simplicity of Lin et al. but absorbs the  $\Theta(1/d)$  equilibrium scaling derived in Eq. (1), so the init-to-equilibrium ratio is  $\text{esi}_k(0)/\text{esi}_k^* = \mathcal{O}(1)$  across widths.

**Empirical validation.** We tested both fixes at  $w=1024$ . With the original  $\rho=0.1$  initialization, the optimum sits at  $\gamma=3.51 \times 10^{-3}$  ( $\sim 2.3\times$  below the base optimum  $8.11 \times 10^{-3}$ ). With a *uniform* smaller  $\rho=0.001$  across all factors (which is neither Option A nor Option B — both options vary  $\lambda_k(0)$  per factor) the optimum moves back to  $\gamma=8.11 \times 10^{-3}$  and the U-curve flattens to a spread of 0.005 nats (vs. 0.07 for the original init). The *principled* Option B with  $\rho_0=1$ ,  $\lambda_k(0)=1/d_k$ , does **not** reproduce this shift: in a full 12-LR Phase 7v2 grid its U-curve tracks the  $\rho=0.1$  baseline almost identically with the same drifted optimum at  $\gamma=3.51 \times 10^{-3}$  (Table 1, last row; Fig. 1). The equilibrium-aligned derivation in Eq. (1) therefore captures *necessary* but not *sufficient* conditions

Table 1: Width-transfer ablation at  $w=1024$  (smoothed test and train  $\hat{L}$ ;  $\alpha = 0.2$ ,  $H = 30$ ). Constant  $\rho=0.1$  favors a downward-drifted peak LR. The empirically-verified *uniform* smaller  $\rho=0.001$  (a special case of neither Option A nor Option B) shifts the optimum back to the base  $\gamma=8.11\times 10^{-3}$ . Option B’s *per-factor*  $\rho_0/d$  with  $\rho_0=1$  does **not** reproduce this shift — its U-curve tracks the baseline almost exactly. A dash (—) indicates the  $\gamma=5.41\times 10^{-3}$  LR was not on the Phase 7v2 grid (only the audit-fix 3-LR sweep covers it).

| $\rho$                            | $\hat{L}_{\text{test}}$ |                      |                      | $\hat{L}_{\text{train}}$ |                      |                      | Best $\gamma_{\text{test}}$            |
|-----------------------------------|-------------------------|----------------------|----------------------|--------------------------|----------------------|----------------------|----------------------------------------|
|                                   | $3.51\times 10^{-3}$    | $5.41\times 10^{-3}$ | $8.11\times 10^{-3}$ | $3.51\times 10^{-3}$     | $5.41\times 10^{-3}$ | $8.11\times 10^{-3}$ |                                        |
| 0.1 (Lin et al.)                  | <b>3.528</b>            | —                    | 3.567                | <b>3.499</b>             | —                    | 3.540                | $3.51\times 10^{-3}$                   |
| 0.01                              | 3.577                   | <b>3.550</b>         | 3.595                | 3.512                    | <b>3.485</b>         | 3.529                | $5.41\times 10^{-3}$                   |
| 0.001                             | 3.566                   | 3.562                | <b>3.562</b>         | 3.500                    | 3.498                | <b>3.497</b>         | <b><math>8.11\times 10^{-3}</math></b> |
| $\rho_0/d_k$ , $\rho_0=1$ (Opt B) | <b>3.577</b>            | —                    | 3.628                | <b>3.512</b>             | —                    | 3.563                | $3.51\times 10^{-3}$                   |

for recovery; the under-tested Option A (data-driven init from the first batch’s  $\text{diag}(\text{GG})$ ) remains the cleanest candidate and has not yet been benchmarked.

**Option B does not reproduce the recovery.** Surprisingly, the principled per-factor scaling  $\lambda_k(0)=\rho_0/d_k$  does *not* restore the base optimum even when  $\rho_0$  is set to 1 — which puts  $\lambda_k(0)=1/d_k$  *below* the uniform  $\rho=0.001$  at the largest factor ( $d_k=2816$  for the MLP-down at  $w=1024$  gives  $\rho_0/d_k \approx 3.6\times 10^{-4}$ ). Option B’s U-curve tracks the  $\rho=0.1$  baseline within 0.04 nats across the whole sweep (Fig. 1, panels at  $\gamma=3.51\times 10^{-3}$  and  $\gamma=8.11\times 10^{-3}$ ) and its minimum sits at  $\gamma=3.51\times 10^{-3}$  alongside the baseline, *not* at the base optimum like uniform  $\rho=0.001$  does. The remaining columns in Table 1 show that uniform  $\rho=0.001$  (“A5b”) is the only configuration that yields a flat U with optimum at the base LR — the equilibrium-aligned per-factor scaling we derived under-predicts the right init for transfer recovery.

**Implementation.** In the reference JAX implementation [klshampoojax], line 4 of Algorithm 2 lives at `_core.py:101` of `kl_shampoo_jax`, where the constant  $\rho^{-1/2}$  is materialized into `eigen_sqrt_inv`. Option A is a one-line edit there: substitute `esi = 1/jnp.sqrt(jnp.diag(GG) + eps)` for `esi = jnp.full(..., 1/jnp.sqrt(init_factor))`, reusing `GG` computed at `_core.py:142--145`.

Zoomed train-loss trajectories at  $w=1024$  (post-warmup) —  $\rho$  comparison

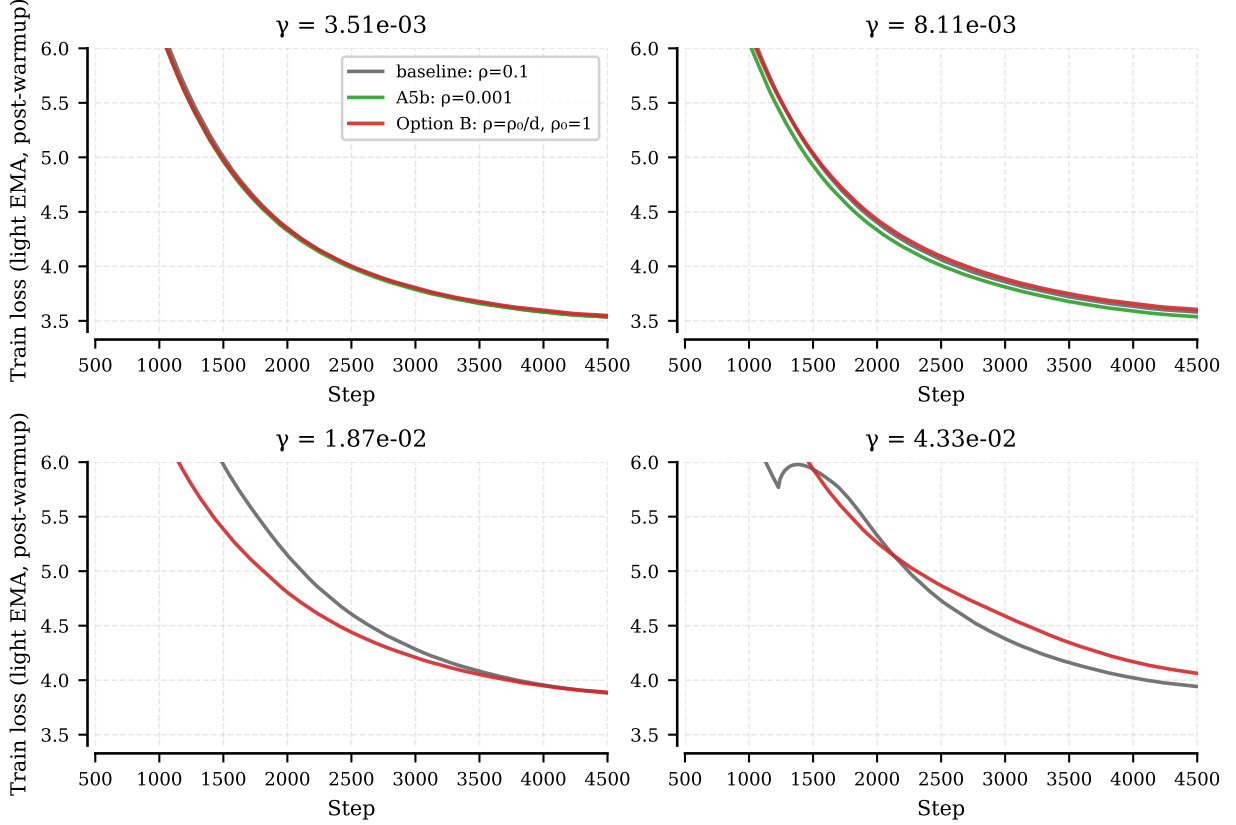


Figure 1: Train-loss trajectories at  $w=1024$  for  $\rho=0.1$  (baseline),  $\rho=0.001$  (uniform; A5b), and Option B ( $\rho_0/d_k$ ,  $\rho_0=1$ ), across four learning rates spanning the optimum and the high-LR instability region. Curves use light EMA ( $\alpha=0.05$ ,  $H=30$ ) and are shown post-warmup (step  $\geq 500$ ). Where  $\rho=0.001$  data was not collected on the Phase 7v2 grid, the trace is omitted. At  $\gamma=3.51\times 10^{-3}$  all three  $\rho$  are indistinguishable; at  $\gamma=8.11\times 10^{-3}$  Option B sits slightly above the others; at  $\gamma=1.87\times 10^{-2}$  Option B’s curve stays consistently above the baseline throughout training; at  $\gamma=4.33\times 10^{-2}$  the baseline ( $\rho=0.1$ ) shows an instability spike around step 1100, while Option B is smoother but ends at higher loss. Option B trades a small loss penalty for slightly better high-LR stability but does **not** shift the optimum.